

Documento Lfa e compiladores

Vitor Inácio Prudente e Guilherme

Tabela de Validações com Expressões Regulares

Campo	Descrição	Expressão Regular (Regex)	Comentário
Placa Mercosul	Formato AAA1A11	<code>^[A-Z]{3}[0-9][A-Z][0-9]{2}\$</code>	3 letras, 1 número, 1 letra, 2 números
CPF	Formato 000.000.000-00	<code>^\d{3}\.\d{3}\.\d{3}-\d{2}\$</code>	Valida apenas o formato
CNPJ	Formato 00.000.000/0000-00	<code>^\d{2}\.\d{3}\.\d{3}/\d{4}-\d{2}\$</code>	Valida apenas o formato
CEP	Formato 00000-000	<code>^\d{5}-\d{3}\$</code>	CEP brasileiro
Telefone	Formato (62) 99999-9999	<code>^\(\d{2}\)\s\d{4,5}-\d{4}\$</code>	Telefone fixo ou celular
E-mail	Email válido	<code>^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\$</code>	Estrutura padrão
Número da ocorrência	Apenas números	<code>^\d+\$</code>	Identificador da ocorrência
Nome do cidadão	Apenas letras e espaços	<code>^[A-Za-zÀ-ÿ\s]+\$</code>	Aceita acentos

Implementação das Expressões Regulares

Validação de Placa Mercosul

```
String regexPlaca = "^[A-Z]{3}[0-9][A-Z][0-9]{2}$";
```

```
if (placa.matches(regexPlaca)) {  
    System.out.println("Placa válida");  
}
```

Explicação

Símbolo	Significado
<code>^</code>	início da string
<code>[A-Z]{3}</code>	três letras
<code>[0-9]</code>	um número
<code>[A-Z]</code>	uma letra
<code>[0-9]{2}</code>	dois números
<code>\$</code>	fim da string

Exemplo válido:

ABC1D23

Validação de CPF

String regexCPF = `"^\\d{3}\\.\\d{3}\\.\\d{3}-\\d{2}$"`;

Exemplo válido:

123.456.789-10

Validação de E-mail

String regexEmail =
`"^[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\\.[A-Za-z]{2,}$"`;

Exemplo válido:

usuario@gmail.com

Autômatos Finitos:

Autômato 1 – Placa Mercosul

Regex:

`[A-Z]{3}[0-9][A-Z][0-9]{2}`

Diagrama:

(q0)

|

L

v

(q1)

|

L

v

(q2)

|

L

v

(q3)

|

N

v

(q4)

|

L

v

(q5)

|

N

v

(q6)

|

N

v

(qF)

Onde:

L = Letra (A-Z)

N = Número (0-9)

qF = Estado Final

Autômato 2 – CPF

Regex:

`\d{3}\.\d{3}\.\d{3}-\d{2}`

Diagrama simplificado:

q0 -> q1 -> q2 -> q3

 | | |
 d d d

q3 --"."--> q4

q4 -> q5 -> q6 -> q7

 d d d

q7 --"."--> q8

q8 -> q9 -> q10 -> q11

 d d d

q11 --"."--> q12

q12 -> q13 -> qF

 d d

Autômato 3 – Email

Regex:

`[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}`

Diagrama simplificado:

q0 -- letra/número --> q1

q1 -- letra/número --> q1

q1 -- @ --> q2

q2 -- letra/número --> q3

q3 -- letra/número --> q3

q3 -- . --> q4

q4 -- letra --> q5

q5 -- letra --> qF

Consulta Avançada

Exemplo de comando:

parametro.tipo = "Acidente" AND parametro.setor = "Sul" OR parametro.status = "Aberto"

Analizador Léxico

Tokens Reconhecidos

Token	Expressão
PARAMETRO	parametro
PONTO	.
IDENTIFICADOR	tipo, setor, status
IGUAL	=
STRING	"texto"
AND	AND
OR	OR
ABREPARE	(

FECHAPAR)

Exemplo

Entrada:

parametro.tipo = "Incêndio" AND parametro.setor = "Norte"

Saída do Léxico:

<PARAMETRO>
<PONTO>
<IDENTIFICADOR,tipo>
<IGUAL>
<STRING,"Incêndio">
<AND>
<PARAMETRO>
<PONTO>
<IDENTIFICADOR,setor>
<IGUAL>
<STRING,"Norte">

Gramática Livre de Contexto

Símbolo inicial:

S

Produções:

$S \rightarrow E$

$E \rightarrow T E'$

$E' \rightarrow \text{AND } T E'$

$E' \rightarrow \text{OR } T E'$

$E' \rightarrow \epsilon$

$T \rightarrow F$

$F \rightarrow \text{CAMPO IGUAL VALOR}$

$\text{CAMPO} \rightarrow \text{PARAMETRO PONTO IDENTIFICADOR}$

$\text{VALOR} \rightarrow \text{STRING}$

onde:

$\text{PARAMETRO} \rightarrow \text{parametro}$

$\text{PONTO} \rightarrow .$

$\text{IGUAL} \rightarrow =$

$\text{STRING} \rightarrow \text{"texto"}$

Derivação

Para a entrada:

`parametro.tipo = "Incêndio" AND parametro.setor = "Norte"`

Derivação:

S

$\Rightarrow E$

$\Rightarrow T E'$

$\Rightarrow F E'$

$\Rightarrow \text{CAMPO} = \text{VALOR } E'$

$\Rightarrow \text{parametro} . \text{tipo} = \text{"Incêndio"} E'$

$\Rightarrow \text{parametro} . \text{tipo} = \text{"Incêndio"} \text{ AND } T E'$

$\Rightarrow \text{parametro} . \text{tipo} = \text{"Incêndio"} \\ \text{AND } F E'$

$\Rightarrow \text{parametro} . \text{tipo} = \text{"Incêndio"} \\ \text{AND } \text{CAMPO} = \text{VALOR } E'$

$\Rightarrow \text{parametro} . \text{tipo} = \text{"Incêndio"}$

AND parametro . setor = "Norte"

$\Rightarrow \varepsilon$

FIRST e FOLLOW

FIRST

$\text{FIRST}(S) = \{ \text{PARAMETRO} \}$

$\text{FIRST}(E) = \{ \text{PARAMETRO} \}$

$\text{FIRST}(E') = \{ \text{AND}, \text{OR}, \varepsilon \}$

$\text{FIRST}(T) = \{ \text{PARAMETRO} \}$

$\text{FIRST}(F) = \{ \text{PARAMETRO} \}$

$\text{FIRST}(\text{CAMPO}) = \{ \text{PARAMETRO} \}$

$\text{FIRST}(\text{VALOR}) = \{ \text{STRING} \}$

FOLLOW

$\text{FOLLOW}(S) = \{ \$ \}$

$\text{FOLLOW}(E) = \{ \$ \}$

$\text{FOLLOW}(E') = \{ \$ \}$

$\text{FOLLOW}(T) = \{ \text{AND}, \text{OR}, \$ \}$

$\text{FOLLOW}(F) = \{ \text{AND}, \text{OR}, \$ \}$

$\text{FOLLOW}(\text{CAMPO}) = \{ \text{IGUAL} \}$

$\text{FOLLOW}(\text{VALOR}) = \{ \text{AND}, \text{OR}, \$ \}$

Tabela Sintática LL(1)

Não Terminal	PARÂMETRO	AND	OR	STRING	\$
S	$S \rightarrow E$				
E	$E \rightarrow T E'$				
E'		$E' \rightarrow$ AND T E'	$E' \rightarrow$ OR T E'		$E' \rightarrow \epsilon$
T	$T \rightarrow F$				
F	$F \rightarrow \text{CAMPO} = \text{VALOR}$				
CAMPO	$\text{CAMPO} \rightarrow \text{PARAMETRO} .$ IDENTIFICADOR				
VALOR				$\text{VALOR} \rightarrow$ STRING	

Técnica Escolhida

Parser Preditivo Descendente (LL(1))

Motivos:

- Fácil implementação em Java/Spring Boot.
- Não possui recursão à esquerda.
- Permite validação rápida dos filtros digitados pelo administrador.
- Adequado para consultas simples utilizando:

AND

OR

=